Traineeship Application

Sprint Report

<1, Αθανάσιος Φυτιλής 5381>

<2, Αντρέας Δημήτριος Καραπέδης 4693>

<3, Γεράσιμος Λιακόπουλος 5156>

VERSIONS HISTORY

Date	Version	Description	Author

1 Introduction

This report documents the design and implementation of a traineeship management web application developed using the Spring Boot framework. The application is part of a university project aiming to provide a complete digital platform for managing internship (traineeship) positions. It supports multiple user roles including students, professors, companies, and members of the traineeship committee, each with different permissions and interfaces tailored to their functionality.

The system was implemented following the Model-View-Controller (MVC) architecture to ensure clean separation of concerns and maintainability. The backend was developed using Java and Spring Boot, while Thymeleaf was used for rendering server-side HTML views. Authentication and role-based authorization were implemented using Spring Security, allowing users to register, log in, and access role-specific dashboards.

For data persistence, the application uses a MySQL relational database managed through Spring Data JPA. Entities such as `User`, `Role`, `Student`, `Company`, `TraineeshipPosition`, and `Evaluation` are mapped using JPA annotations. The project also provides experience with configuring MySQL in a Spring environment, defining entity relationships, and using repositories to handle data operations.

The development was structured in sprints following Scrum methodology, focusing initially on core features like user authentication and registration, and gradually expanding to cover more complex business logic related to traineeship matching and evaluation.

1.1 Purpose

The purpose of this project is to simulate the implementation of a real-world enterprise-grade application that facilitates the management of traineeship positions at a university level. It helps all involved stakeholders—students, professors, companies, and the committee—to digitally perform actions such as registering, announcing or applying for positions, evaluating performance, and assigning supervisors.

From an educational standpoint, the project aims to help students become familiar with modern software engineering practices and frameworks. Through this assignment, the student gains hands-on experience in:

- Developing secure web applications using Java and Spring Boot,
- Setting up and configuring a MySQL database,
- Applying MVC and layered architecture patterns,
- Implementing real-world use cases with role-specific flows,

- Writing clean, maintainable, and testable code in a team-oriented environment.

The project also highlights the importance of modular design and extensibility, as the application is built to accommodate future strategies for matching students to positions or professors to students, using design patterns like Strategy and Factory.

2 Scrum team and Sprint Backlog

2.1 Scrum team

Product Owner	THANASIS
Scrum Master	THANASIS
Development Team	THANASIS,ANTREAS,GERASIMOS

2.2 Sprints

<List below the sprints that you performed and the user stories that have been realized in each Sprint>

Sprint No	Begin Date	End Date	Number of weeks	User stories
Sprint 1	**04/03/2025**	**17/03/2025**	**2**	**US1, US2,US3**
Sprint 2	**18/03/2025**	**14/04/2025**	**4**	**US4,US7,US8,US10,US5,US6,US16, US17, US18, US9, US11, US12**
Sprint 3	**14/04/2025**	**05/05/2025**	**3**	**US13,US14,US15,US19,US20,US21**

3 Use Cases

<Specify the concrete Use Cases that describe the interaction of the user with the applications, as derived from the abstract user stories. Give a **UML Use Case diagram** and the **detailed use case descriptions**.>

3.1 <Use Case 1>

Use case ID	UC1
Actors	USER (STUDENT, PROFESSOR, COMPANY, COMMITTEE)
Pre conditions	The user does not already have an account in the system. The registration form is accessible.
Main flow of events	1) The user clicks on the "Register" option from the home page. 2) The system displays the registration form. 3) The user fills in all required fields: username, password, full name, email, and role (STUDENT, PROFESSOR, COMPANY, COMMITTEE). 4)The system validates: • That the username is not already taken. • That all required fields are properly filled and valid. 5) The AuthController forwards the registration data to AuthService. 6) AuthServiceImpl creates the corresponding entity (Student, Professor, etc.) and saves it to the database. 7) A success message is shown and the user is redirected to the login page.
Alternative flow 1	IF THE USERNAME IS ALREADY IN USE, THE SYSTEM SHOWS AN ERROR AND PROMPTS THE USER TO CHOOSE A DIFFERENT ONE.
Alternative flow 2	IF ANY FIELD IS INVALID (E.G., EMPTY OR INCORRECT FORMAT), THE SYSTEM DISPLAYS AN ERROR AND ASKS THE USER TO CORRECT THE INPUT.
Post conditions	A NEW USER ACCOUNT IS SUCCESSFULLY CREATED AND STORED IN THE APPROPRIATE TABLE (BASED ON THE ROLE). THE USER IS ABLE TO PROCEED TO LOGIN.

<Use Case 2>

Use case ID	UC2
Actors	USER (STUDENT, PROFESSOR, COMPANY, COMMITTEE)
Pre conditions	THE USER HAS A REGISTERED ACCOUNT WITH VALID CREDENTIALS. THE LOGIN FORM IS ACCESSIBLE.

Main flow of events	1. The user navigates to the login page.
	2. The system displays the login form requesting username and password.
	3. The user fills in the credentials and submits the form.
	4. The system forwards the credentials to Spring Security for authentication.
	5. If the credentials are valid:
	  o The user is logged in.
	  o The system redirects the user to the appropriate dashboard based on their role.
	6. A welcome message or dashboard is displayed.

Alternative flow 1	IF THE CREDENTIALS ARE INCORRECT, THE SYSTEM DISPLAYS AN ERROR MESSAGE AND REMAINS ON THE LOGIN PAGE.
Alternative flow 2	IF THE USER ACCOUNT IS DISABLED OR INACTIVE, THE SYSTEM DENIES LOGIN AND INFORMS THE USER ACCORDINGLY.
Post conditions	THE USER IS SUCCESSFULLY AUTHENTICATED AND GRANTED ACCESS TO THEIR ROLE-BASED DASHBOARD. A SESSION IS CREATED FOR THE USER.

<Use Case 3>

Use case ID	UC3
Actors	USER (STUDENT, PROFESSOR, COMPANY, COMMITTEE)
Pre conditions	THE USER IS AUTHENTICATED (LOGGED IN). THE USER'S PROFILE INFORMATION EXISTS IN THE DATABASE.
Main flow of events	1. THE USER LOGS INTO THE SYSTEM AND ACCESSES THE DASHBOARD. 2. THE USER SELECTS THE "VIEW PROFILE" OPTION FROM THE NAVIGATION MENU. 3. THE CONTROLLER RETRIEVES THE USER'S PROFILE DATA BASED ON THE LOGGED-IN USERNAME. 4. THE PROFILE DATA IS LOADED FROM THE APPROPRIATE TABLE (BASED ON THE USER'S ROLE). 5. THE PROFILE INFORMATION IS POPULATED INTO THE PROFILE VIEW. 6. THE SYSTEM DISPLAYS THE USER'S PROFILE PAGE.
Alternative flow 1	IF THE PROFILE DATA CANNOT BE FOUND, THE SYSTEM REDIRECTS THE USER TO AN ERROR PAGE OR SHOWS A MESSAGE.
Post conditions	THE USER SUCCESSFULLY VIEWS THEIR PROFILE DETAILS.

<Use Case 4>

Use case ID	UC4
Actors	STUDENT
Pre conditions	THE STUDENT IS REGISTERED AND LOGGED IN. NO STUDENT PROFILE EXISTS YET IN THE SYSTEM FOR THIS USER.
Main flow of events	1. THE STUDENT LOGS INTO THE SYSTEM. 2. THE STUDENT NAVIGATES TO THE "PROFILE" SECTION AND SELECTS "CREATE PROFILE". 3. The system displays a form requiring: • Full Name • University ID • List of Interests (e.g., Java, AI) • List of Skills (e.g., Python, SQL) • Preferred Location (e.g., Athens, Thessaloniki) 4. The student fills in the form and submits it. 5. The system validates all fields. 6. If the data is valid, the information is passed to StudentController and handled by StudentServiceImpl. 7. The service stores the student profile in the database via StudentDAO. 8. A confirmation message is shown and the student is redirected to the profile view.
Alternative flow 1	IF REQUIRED FIELDS ARE MISSING OR INVALID (E.G., NON-NUMERIC UNIVERSITY ID), THE SYSTEM DISPLAYS AN ERROR AND REDISPLAYS THE FORM WITH PREVIOUSLY ENTERED VALUES.
Alternative flow 2	IF THE STUDENT HAS ALREADY CREATED A PROFILE, THE SYSTEM PREVENTS CREATION AND REDIRECTS TO THE PROFILE VIEW OR UPDATE SCREEN.
Post conditions	A STUDENT PROFILE IS CREATED AND STORED IN THE DATABASE. THE DATA BECOMES AVAILABLE FOR THE TRAINEESHIP MATCHING STRATEGIES.

<Use Case 5>

Use case ID	UC5
Actors	STUDENT
Pre conditions	THE STUDENT IS LOGGED IN. THE STUDENT HAS A VALID AND COMPLETE PROFILE IN THE SYSTEM. THERE ARE AVAILABLE TRAINEESHIP POSITIONS LISTED.
Main flow of events	1. THE STUDENT LOGS INTO THE SYSTEM AND ACCESSES THE DASHBOARD. 2. THE STUDENT NAVIGATES TO THE "AVAILABLE POSITIONS" PAGE. 3. THE SYSTEM DISPLAYS A LIST OF OPEN TRAINEESHIP POSITIONS. 4. THE STUDENT SELECTS A POSITION AND CLICKS "APPLY". 5. THE STUDENTCONTROLLER CALLS THE APPLICATIONSERVICEIMPL TO HANDLE THE REQUEST. 6. THE SERVICE CHECKS IF THE STUDENT HAS ALREADY APPLIED FOR THIS POSITION. 7. IF NOT, A NEW APPLICATION ENTITY IS CREATED AND SAVED VIA APPLICATIONDAO. 8. THE SYSTEM CONFIRMS THAT THE APPLICATION WAS SUCCESSFUL.
Alternative flow 1	IF THE STUDENT HAS ALREADY APPLIED FOR THE SELECTED POSITION, THE SYSTEM PREVENTS DUPLICATION AND DISPLAYS AN INFORMATIVE MESSAGE.
Alternative flow 2	IF SAVING THE APPLICATION FAILS (E.G. DATABASE ERROR), AN ERROR MESSAGE IS SHOWN AND THE ACTION IS ABORTED
Post conditions	A NEW APPLICATION ENTRY IS STORED IN THE DATABASE LINKING THE STUDENT TO THE SELECTED TRAINEESHIP POSITION. THE STUDENT MAY SEE THEIR APPLICATION IN THE "MY APPLICATIONS"

	SECTION.

<Use Case 6>

Use case ID	UC6
Actors	STUDENT
Pre conditions	• The student is logged in. • The student has been assigned a traineeship position. • The student has access to the logbook submission interface.
Main flow of events	1. The student logs into the system and accesses the "Logbook" section from their dashboard. 2. The system displays the current list of logbook entries (if any). 3. The student clicks on “Add Entry” or similar action. 4. The system displays a form for entering logbook details (e.g., activity description, date, time spent, notes). 5. The student completes the form and submits it. 6. The system validates the input and associates the logbook entry with the current student and their assigned traineeship position. 7. The entry is saved to the database via the LogbookDAO. 8. A confirmation message is shown and the new entry is listed in the logbook view.
Alternative flow 1	IF THE STUDENT HAS NOT BEEN ASSIGNED A POSITION YET, THE SYSTEM PREVENTS LOGBOOK ENTRY AND INFORMS THE USER.
Alternative flow 2	IF ANY REQUIRED FIELD IS MISSING OR INVALID, THE SYSTEM DISPLAYS AN ERROR AND RE-DISPLAYS THE FORM FOR CORRECTION.
Post	A NEW LOGBOOK ENTRY IS STORED IN THE DATABASE LINKED TO THE STUDENT

conditions	AND THEIR ASSIGNED TRAINEESHIP POSITION. THE ENTRY IS VISIBLE IN THE STUDENT'S LOGBOOK HISTORY.

<Use Case 7>

Use case ID	UC7
Actors	COMPANY
Pre conditions	THE COMPANY USER IS REGISTERED AND LOGGED IN. NO PROFILE EXISTS YET IN THE DATABASE.
Main flow of events	1. THE COMPANY LOGS IN AND IS REDIRECTED TO ITS DASHBOARD. 2. THE SYSTEM DETECTS A SEMI-EMPTY PROFILE AND REDIRECTS THE COMPANY TO THE "UPDATE PROFILE" FORM. 3. The company fills out the following fields: • fullName: The name of the company or its contact person. • location: Preferred location of traineeships. 4. The user submits the form. 5. The CompanyController.updateProfile(...) method receives the data. 6. The controller passes it to CompanyServiceImpl.updateProfile(...). 7. The service persists the updated Company object using CompanyDAO. 8. A success message is shown and the updated profile is displayed.
Alternative flow 1	IF FULLNAME OR OTHER REQUIRED FIELDS ARE MISSING OR INVALID, THE FORM IS REDISPLAYED WITH AN ERROR MESSAGE.
Alternative flow 2	IF THE PROFILE ALREADY EXISTS, THE SYSTEM ALLOWS ONLY UPDATING, NOT RE-CREATING.
Post conditions	• THE COMPANY'S PROFILE IS STORED OR UPDATED IN THE DATABASE. • THE COMPANY CAN NOW CREATE TRAINEESHIP POSITIONS.

<Use Case 8>

Use case ID	UC8
Actors	COMPANY
Pre conditions	• The company is logged in. • The company has previously created at least one traineeship position. • The traineeship positions are in the "AVAILABLE" status.
Main flow of events	1. The company logs into the system and accesses the company dashboard. 2. From the dashboard, the company selects “traineeships” or a similar link. 3. The system retrieves the list of all TraineeshipPosition records created by the logged-in company where the status is AVAILABLE. 4. The positions are fetched from the database using TraineeshipPositionDAO.findBySupervisorUsername(...) or similar repository query. 5. The list of positions is displayed in the UI with basic details: title, location, description, start/end date.
Alternative flow 1	IF THERE ARE NO POSITIONS IN AVAILABLE STATUS, THE SYSTEM INFORMS THE COMPANY WITH A MESSAGE SUCH AS "NO ACTIVE POSITIONS AVAILABLE."
Alternative flow 2	IF THE COMPANY IS NOT AUTHENTICATED, ACCESS IS DENIED AND THE USER IS REDIRECTED TO THE LOGIN PAGE.
Post conditions	THE COMPANY SUCCESSFULLY VIEWS A LIST OF ALL ITS ACTIVE AND AVAILABLE TRAINEESHIP POSITIONS.

<Use Case 9>

Use case ID	UC9
Actors	COMPANY
Pre conditions	• The company is logged into the system. • At least one of the traineeship positions created by the company has been assigned to a student (status: ASSIGNED).
Main flow of events	1. The company logs in and accesses its dashboard. 2. From the dashboard, the company selects the "Assigned Students" or equivalent option. 3. The system retrieves all TraineeshipPosition records created by this company with status ASSIGNED. 4. The controller uses TraineeshipPositionDAO.findBySupervisorUsername(...) or similar to fetch the positions. 5. For each position, the system displays: • Position details (title, location, etc.) • Assigned student's full name and contact (if available) 6. The company views the list and can proceed to evaluate the student later.
Alternative flow 1	IF NO POSITIONS HAVE BEEN ASSIGNED, THE SYSTEM INFORMS THE COMPANY: "NO STUDENTS HAVE BEEN ASSIGNED YET."
Alternative flow 2	IF THE USER IS NOT LOGGED IN AS A COMPANY, THE REQUEST IS DENIED AND REDIRECTED TO LOGIN.
Post conditions	THE COMPANY SUCCESSFULLY VIEWS THE LIST OF ASSIGNED TRAINEESHIP POSITIONS WITH THE ASSOCIATED STUDENTS.

<Use Case 10>

Use case ID	UC10
Actors	COMPANY

Pre conditions	THE COMPANY IS LOGGED INTO THE SYSTEM. THE COMPANY HAS ALREADY CREATED ITS COMPANY PROFILE. THE COMPANY HAS ACCESS TO THE "ADD POSITION" FORM.
Main flow of events	1. THE COMPANY LOGS IN AND ACCESSES ITS DASHBOARD. 2. THE COMPANY SELECTS “MAKE A TRAINEESHIP”. 3. The system displays a form for a new traineeship position, including: • Description • Start and End Date • Location • Skills Required 4. The company fills out and submits the form. 5. The CompanyController.savePosition(...) receives the submitted data. 6. The controller delegates to CompanyServiceImpl.createTraineeshipPosition(...). 7. The service saves the position using TraineeshipPositionDAO. 8. The new position is saved with status AVAILABLE. 9. The system displays a success message and shows the list of created positions.
Alternative flow 1	IF REQUIRED FIELDS ARE MISSING OR INVALID (E.G., END DATE BEFORE START DATE), THE SYSTEM SHOWS AN ERROR AND RETURNS TO THE FORM WITH VALIDATION MESSAGES.
Alternative flow 2	IF THE COMPANY IS NOT AUTHENTICATED, THE REQUEST IS REDIRECTED TO THE LOGIN PAGE.
Post conditions	A NEW TRAINEESHIP POSITION IS STORED IN THE DATABASE AND BECOMES VISIBLE TO STUDENTS AS “AVAILABLE”

<Use Case 11>

Use case ID	UC11

Actors	COMPANY
Pre conditions	• The company is logged in. • The company has created at least one traineeship position. • The traineeship position to be deleted is already assigned and completed.
Main flow of events	1. THE COMPANY LOGS INTO THE SYSTEM AND NAVIGATES TO THE "ASSIGNED STUDENTS" SECTION. 2. The system displays all positions created by the company. 3. The company selects a position to delete and clicks the “Delete” action. 4. The CompanyController receives the delete request and verifies company ownership. 5. The controller invokes CompanyServiceImpl.deletePositionById(positionId) or equivalent. 6. The service uses TraineeshipPositionDAO.deleteById() to remove the position from the database. 7. A confirmation message is shown and the list is refreshed.
Alternative flow 1	IF DELETION FAILS DUE TO A DATABASE ERROR, AN ERROR MESSAGE IS DISPLAYED AND THE ACTION IS LOGGED.
Post conditions	THE TRAINEESHIP POSITION IS REMOVED FROM THE DATABASE. IT IS NO LONGER VISIBLE TO STUDENTS OR COMMITTEE MEMBERS.

<Use Case 12>

Use case ID	UC12
Actors	COMPANY

Pre conditions	THE COMPANY IS LOGGED INTO THE SYSTEM. THE STUDENT HAS BEEN OFFICIALLY ASSIGNED TO A TRAINEESHIP POSITION CREATED BY THE COMPANY. THE TRAINEESHIP HAS BEEN MARKED AS COMPLETED OR IS PAST ITS END DATE.
Main flow of events	1. THE COMPANY LOGS INTO THE SYSTEM AND NAVIGATES TO THE "ASSIGNED STUDENTS" SECTION. 2. THE SYSTEM DISPLAYS ALL STUDENTS ASSIGNED TO POSITIONS CREATED BY THE COMPANY. 3. THE COMPANY SELECTS A STUDENT AND CLICKS ON “EVALUATE”. 4. The system displays an evaluation form with fields such as: • Evaluation text/notes • Numeric score (e.g. motivation, effectiveness and efficiency 1 to 5) • Date 5. The company fills in the form and submits the evaluation. 6. CompanyController.submitEvaluation(...) handles the request. 7. The request is passed to EvaluationServiceImpl.submitCompanyEvaluation(...). 8. The evaluation is persisted using EvaluationDAO, linked to both the student and the position. 9. A confirmation message is shown.
Alternative flow 1	IF THE STUDENT IS NOT ASSIGNED TO THIS COMPANY, THE EVALUATION FORM IS NOT ACCESSIBLE.
Alternative flow 2	IF THE COMPANY HAS ALREADY SUBMITTED AN EVALUATION FOR THE SAME STUDENT AND POSITION, THE SYSTEM UPDATES THE EVALUATION .
Post conditions	A NEW EVALUATION ENTRY IS STORED IN THE DATABASE ASSOCIATED WITH THE STUDENT AND THE TRAINEESHIP POSITION. THE EVALUATION IS ACCESSIBLE BY COMMITTEE MEMBERS FOR FINAL REVIEW.

<Use Case 13>

Use case ID	UC13
Actors	PROFESSOR
Pre conditions	THE PROFESSOR IS REGISTERED AND LOGGED INTO THE SYSTEM. NO PROFILE DATA (FULL NAME, INTERESTS) IS CURRENTLY SET FOR THIS USER.
Main flow of events	1. THE PROFESSOR LOGS IN AND IS REDIRECTED TO THE DASHBOARD. 2. THE PROFESSOR SELECTS "UPDATE YOUR PROFILE". 3. THE SYSTEM DETECTS A MISSING OR INCOMPLETE PROFILE AND SHOWS THE PROFILE CREATION FORM. 4. The form includes fields such as: • Full Name • Interests (comma-separated or list input) 5. The professor fills out the form and submits it. 6. The request is handled by ProfessorController.saveProfile(...). 7. The controller sends the data to ProfessorServiceImpl.updateProfile(...). 8. The service updates the corresponding Professor entity and persists it via ProfessorDAO. 9. A confirmation message is shown and the updated profile is displayed.
Alternative flow 1	IF FULLNAME IS EMPTY OR INTERESTS ARE NOT VALID, THE SYSTEM SHOWS AN ERROR MESSAGE AND REDISPLAYS THE FORM.
Post conditions	THE PROFESSOR'S PROFILE (NAME AND INTERESTS) IS SAVED IN THE DATABASE. IT IS USED LATER FOR TRAINEESHIP SUPERVISOR ASSIGNMENT.

<Use Case 14>

Use case ID	UC14
Actors	PROFESSOR
Pre conditions	THE PROFESSOR IS LOGGED INTO THE SYSTEM. THE PROFESSOR HAS BEEN ASSIGNED TO ONE OR MORE TRAINEESHIP POSITIONS AS SUPERVISOR. THE POSITIONS ARE IN PROGRESS OR ACTIVE IN THE SYSTEM.
Main flow of events	1. THE PROFESSOR LOGS INTO THE SYSTEM. 2. FROM THE DASHBOARD, THE PROFESSOR SELECTS THE “MY SUPERVISED POSITIONS” SECTION. 3. THE SYSTEM RETRIEVES ALL TRAINEESHIPPOSITION ENTRIES WHERE THE LOGGED-IN PROFESSOR IS LISTED AS THE SUPERVISOR. 4. THE DATA IS FETCHED USING TRAINEESHIPPOSITIONDAO.FINDBYSUPERVISOR(…) OR SIMILAR METHOD. 5. The system displays the list of supervised traineeship positions, including: • Title, location, assigned student (if any), start/end date, status.
Alternative flow 1	IF THE PROFESSOR DOES NOT SUPERVISE ANY POSITION, THE SYSTEM DISPLAYS A MESSAGE: “YOU ARE NOT CURRENTLY ASSIGNED TO ANY TRAINEESHIP.”
Alternative flow 2	IF THE USER IS NOT A PROFESSOR OR IS NOT AUTHENTICATED, ACCESS IS DENIED AND THE USER IS REDIRECTED TO THE LOGIN SCREEN.
Post conditions	THE PROFESSORS ARE ABLE TO VIEW A LIST OF ALL TRAINEESHIP POSITIONS THEY SUPERVISE.

<Use Case 15>

Use case ID	UC15
Actors	PROFESSOR

Pre conditions	THE PROFESSOR IS LOGGED IN. THE PROFESSOR IS ASSIGNED AS SUPERVISOR TO AT LEAST ONE TRAINEESHIP POSITION. THE ASSIGNED STUDENT HAS STARTED OR COMPLETED THEIR TRAINEESHIP.
Main flow of events	1. THE PROFESSOR LOGS INTO THE SYSTEM AND NAVIGATES TO "SUPERVISED POSITIONS". 2. THE SYSTEM DISPLAYS ALL POSITIONS THE PROFESSOR SUPERVISES WITH THE ASSOCIATED STUDENTS. 3. THE PROFESSOR SELECTS A STUDENT/POSITION AND CLICKS "EVALUATE". 4. The system displays an evaluation form including: • Motivation rating (1 to 5) • Effectiveness rating (1 to 5) • Efficiency rating (1 to 5) • Hosting Company rating: • Optional comments 5. The professor submits the form. 6. ProfessorController.submitEvaluation(...) handles the request. 7. The controller invokes EvaluationService.saveEvaluation(...), which: • Validates evaluator role and position • Creates or updates the Evaluation entry with EvaluatorType.PROFESSOR • Saves via EvaluationDAO 8. The system confirms the submission and redirects to the evaluations list.
Alternative flow 1	IF THE PROFESSOR IS NOT THE SUPERVISOR OF THE SELECTED POSITION, THE EVALUATION IS BLOCKED WITH AN ACCESS DENIED MESSAGE.
Alternative flow 2	IF REQUIRED FIELDS ARE MISSING OR OUT OF RANGE, THE SYSTEM RETURNS ERRORS AND RELOADS THE FORM WITH PRIOR INPUT.
Post conditions	• AN EVALUATION IS STORED IN THE SYSTEM, LINKED TO THE STUDENT AND THE TRAINEESHIP POSITION, WITH EVALUATOR TYPE PROFESSOR.

<Use Case 16>

Use case ID	UC16
Actors	COMMITTEE MEMBER
Pre conditions	• The committee member is authenticated in the system. • At least one student application has been submitted for traineeship positions.
Main flow of events	1. The committee member logs into the system. 2. From the dashboard, they select “Your Applications”. 3. The system queries all applications submitted by students. 4. CommitteeController.viewAllApplications(...) calls the appropriate service method. 5. ApplicationServiceImpl.getAllApplications(...) fetches all Application entries from the database using ApplicationDAO. 6. For each application, the system displays: • Student full name • Desired position(s) • Application date • Current application status (PENDING, ACCEPTED, REJECTED)
Alternative flow 1	IF NO APPLICATIONS EXIST, THE SYSTEM DISPLAYS: “NO STUDENT APPLICATIONS HAVE BEEN SUBMITTED YET.”
Alternative flow 2	IF A NON-COMMITTEE USER TRIES TO ACCESS THIS PAGE, THEY ARE SHOWN AN ERROR.
Post conditions	THE COMMITTEE MEMBER SUCCESSFULLY VIEWS A LIST OF ALL APPLICATIONS SUBMITTED BY STUDENTS.

Use case ID	UC17
Actors	COMMITTEE MEMBER
Pre conditions	• The committee member is logged in. • The student has submitted at least one traineeship application. • The student has a completed profile with interests, skills, and preferred location. • There are available traineeship positions in the system.
Main flow of events	1. The committee member logs in and navigates to the list of student applications. 2. The committee selects a student from the list. 3. The system displays a search interface allowing filtering by: • Interests • Preferred location • Both 4. The committee selects the desired filter strategy. 5. The system applies the chosen search strategy to match available TraineeshipPosition entries to the student's profile. • Using strategies like: ○ similarity on interests ○ Location match ○ Combined 6. The matching positions are fetched from TraineeshipPositionDAO using strategy objects and displayed to the committee. 7. The committee member reviews the results for potential assignment.
Alternative flow 1	IF THE STUDENT HAS NOT FILLED OUT THEIR PROFILE, THE SYSTEM BLOCKS THE SEARCH AND SHOWS AN APPROPRIATE WARNING.
Alternative	IF A STRATEGY IMPLEMENTATION FAILS, THE SYSTEM LOGS THE ERROR AND

flow 2	NOTIFIES THE COMMITTEE OF TECHNICAL ISSUES.
Post conditions	A FILTERED LIST OF MATCHING TRAINEESHIP POSITIONS IS SHOWN TO THE COMMITTEE. THIS LIST CAN BE USED FOR THE CORRECT ASSIGNMENT.

<Use Case 18>

Use case ID	UC18
Actors	COMMITTEE MEMBER
Pre conditions	THE COMMITTEE MEMBER IS LOGGED IN. THE STUDENT HAS SUBMITTED AN APPLICATION AND HAS A COMPLETE PROFILE. A LIST OF AVAILABLE AND SUITABLE TRAINEESHIP POSITIONS IS ALREADY DISPLAYED (VIA UC17). NO OTHER POSITION HAS BEEN ASSIGNED TO THIS STUDENT YET.
Main flow of events	1. THE COMMITTEE MEMBER NAVIGATES TO THE MATCHED POSITIONS LIST FOR A STUDENT (UC17). 2. From the dashboard, they select “Assign Supervisors”. 3. THE COMMITTEE SELECTS A TRAINEESHIP POSITION AND CLICKS “ASSIGN”. 4. THE REQUEST IS SENT TO COMMITTEECONTROLLER.ASSIGNSUPERVISOR(...) OR SIMILAR METHOD. 5. The system verifies: 6. The student is not already assigned. 7. The position is still AVAILABLE. 8. The controller calls ApplicationService.assignStudentToPosition(...) or CommitteeService.assignPosition(...) passing the student ID and position ID.

	9. The service: 10. Updates the Application record to ACCEPTED 11. Updates the TraineeshipPosition status to ASSIGNED 12. Sets the Student field in the TraineeshipPosition 13. A success message is displayed.
Alternative flow 1	IF THE STUDENT ALREADY HAS AN ASSIGNED POSITION, THE ACTION IS BLOCKED.
Alternative flow 2	IF AN ERROR OCCURS WHILE SAVING CHANGES TO THE DATABASE, THE SYSTEM INFORMS THE USER AND LOGS THE ERROR.
Post conditions	• THE SELECTED TRAINEESHIP POSITION IS LINKED TO THE STUDENT. • THE POSITION IS NO LONGER AVAILABLE TO OTHERS. • THE STUDENT SEES THE ASSIGNED POSITION ON THEIR DASHBOARD.

<Use Case 19>

Use case ID	UC19
Actors	COMMITTEE MEMBER
Pre conditions	• The committee member is authenticated. • There exists at least one traineeship position in ASSIGNED status without an assigned professor. • Professors have created their profiles and are eligible for supervision.
Main flow of events	1. The committee logs into the system and views the list of in-progress traineeships without a supervisor. 2. The committee selects a specific traineeship position to allocate a professor. 3. The system displays available professors, allowing the committee to: • Apply the **Interest Matching Strategy** (based on Jaccard similarity

	between professor and position topics) • Apply the **Minimum Load Strategy** (select the professor with the least assigned traineeships) 4. The committee selects the desired strategy and applies it. 5. The system retrieves and ranks suitable professors accordingly. 6. The committee selects a professor from the results and confirms the assignment. 7. The assignment is handled by CommitteeController.assignSupervisor(...). 8. The TraineeshipPosition is updated to include the selected professor via TraineeshipPositionDAO. 9. A success message is displayed.
Alternative flow 1	IF NO PROFESSOR MATCHES THE STRATEGY CRITERIA, THE SYSTEM DISPLAYS AN APPROPRIATE MESSAGE.
Alternative flow 2	IF THE STRATEGY CANNOT BE APPLIED DUE TO DATA OR TECHNICAL ERROR, AN ERROR MESSAGE IS SHOWN.
Post conditions	• THE SELECTED PROFESSOR IS ASSIGNED TO THE TRAINEESHIP POSITION. • THE ASSIGNMENT IS STORED IN THE SYSTEM AND USED FOR EVALUATION AND MONITORING

<Use Case 20>

Use case ID	UC20
Actors	PROFESSOR
Pre conditions	• THE PROFESSOR IS LOGGED IN. • AT LEAST ONE TRAINEESHIP POSITION IS ASSIGNED TO THE PROFESSOR. • THE POSITION'S STATUS IS ASSIGNED.

Main flow of events	1. The professor logs into the system and accesses the “Assigned Positions” section. 2. The system fetches all TraineeshipPosition entries where: • supervisor.username = currentProfessorUsername • status = IN_PROGRESS 3. The controller delegates the request to ProfessorService.getAssignedPositions(...). 4. The service queries the database using TraineeshipPositionDAO.findBySupervisorAndStatus(...). 5. The system displays a list of all active supervised positions including: • Title • Assigned student (if any) • Evaluation average(optional) • Company • City
Alternative flow 1	IF NO RELEVANT POSITIONS ARE FOUND, THE SYSTEM SHOWS: “YOU ARE NOT MANAGE ANY ACTIVE TRAINEESHIP AT THIS TIME.”
Alternative flow 2	.....
Post conditions	• THE COMMITTEE VIEWS ALL ACTIVE TRAINEESHIP POSITIONS IT MANAGE. • IT CAN EXPLOIT LATER EVALUATE STUDENTS FROM THIS VIEW.

<Use Case 21>

Use case ID	UC21
Actors	COMMITTEE MEMBER

Pre conditions	• THE COMMITTEE MEMBER IS AUTHENTICATED. • AT LEAST ONE TRAINEESHIP POSITION IS CURRENTLY IN ASSINED STATE. • BOTH COMPANY AND PROFESSOR EVALUATIONS EXIST FOR THAT TRAINEESHIP.
Main flow of events	1. The committee logs into the system and selects the "ASSIGNED Traineeships" section. 2. A list of traineeship positions with ASSIGNED status is displayed. 3. The committee selects a specific traineeship to review. 4. The system displays evaluation summaries for that position: • Company evaluation (motivation, effectiveness, efficiency) • Professor evaluation (same fields + hosting company ratings) 5. The system presents the overall evaluation results in a summarized format. 6. The committee selects “Complete Traineeship” and chooses a result: • PASS • FAIL 7. The result is submitted through CommitteeController.completeTraineeship(...). 8. The position’s status is updated to COMPLETED, and a final grade is stored. 9. A confirmation message is displayed.
Alternative flow 1	IF EITHER COMPANY OR PROFESSOR EVALUATION IS MISSING, THE SYSTEM BLOCKS FINALIZATION AND SHOWS AN EMPTY BOX.
Alternative flow 2	IF SAVING THE FINAL STATUS FAILS, AN ERROR IS SHOWN AND LOGGED.
Post conditions	• THE TRAINEESHIP POSITION IS MARKED AS COMPLETED. • THE FINAL EVALUATION DECISION (PASS/FAIL) IS STORED AND VISIBLE IN THE SYSTEM. • THE STUDENT’S TRAINEESHIP PROCESS IS OFFICIALLY CLOSED.

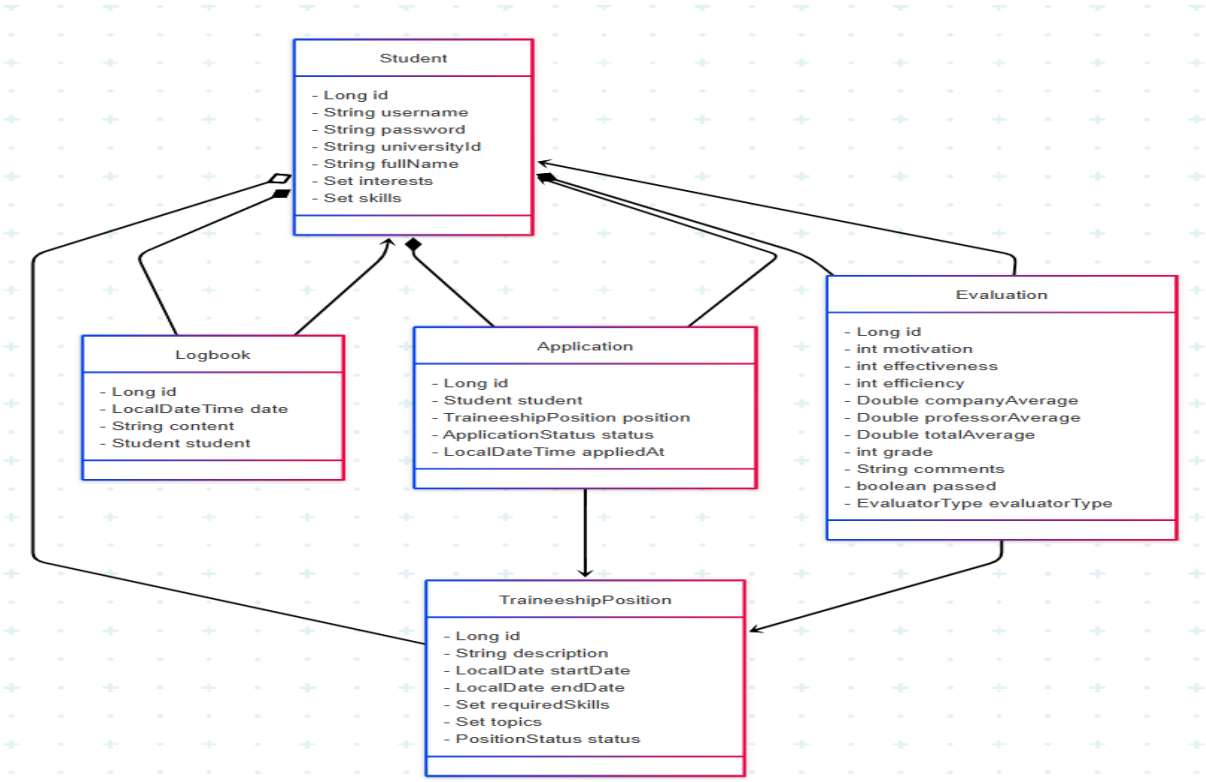
4 Design

4.1 Architecture

<Specify the overall architecture for this release in terms of a **UML package diagram**.>

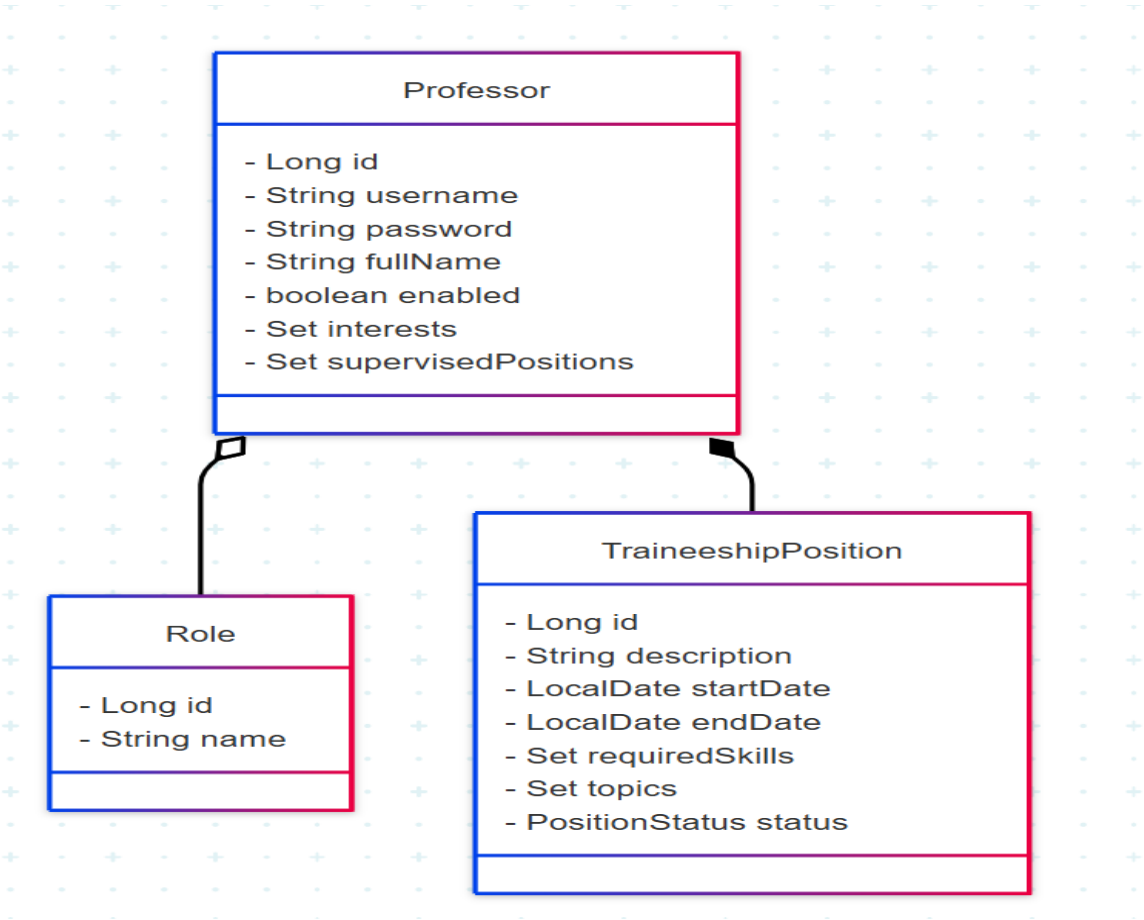
4.2 Design

Model UML'S

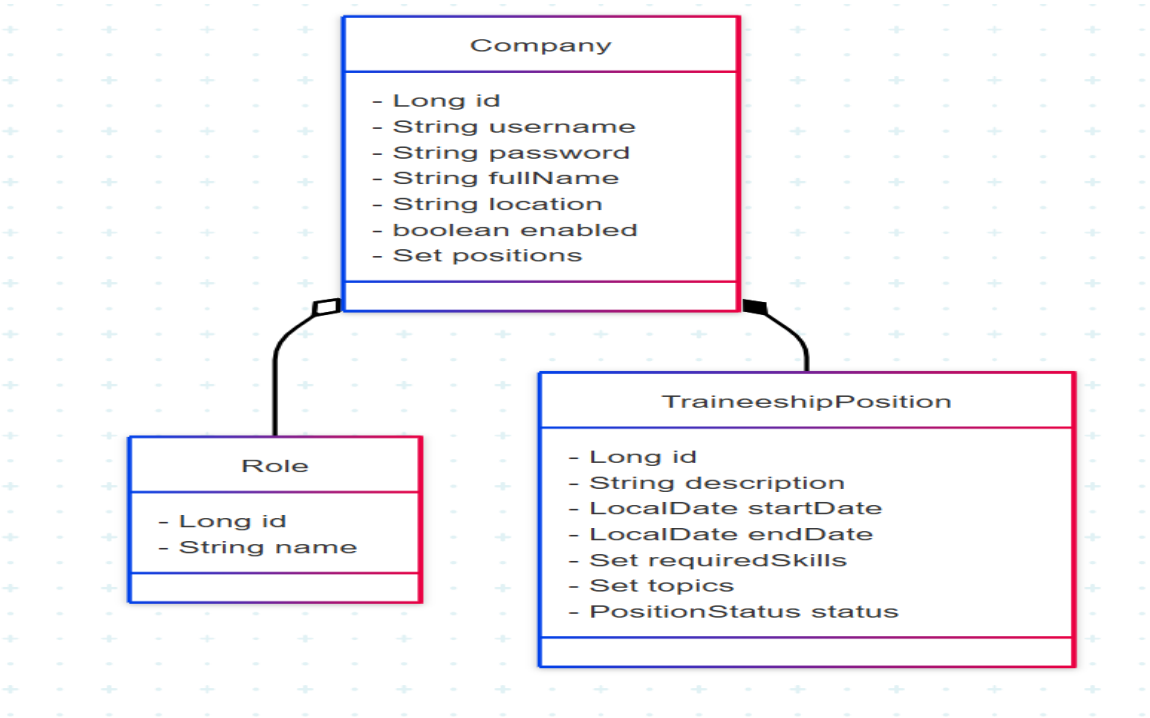


<Document the classes that are included in this release in terms of CRC cards according to the template that is given below.>

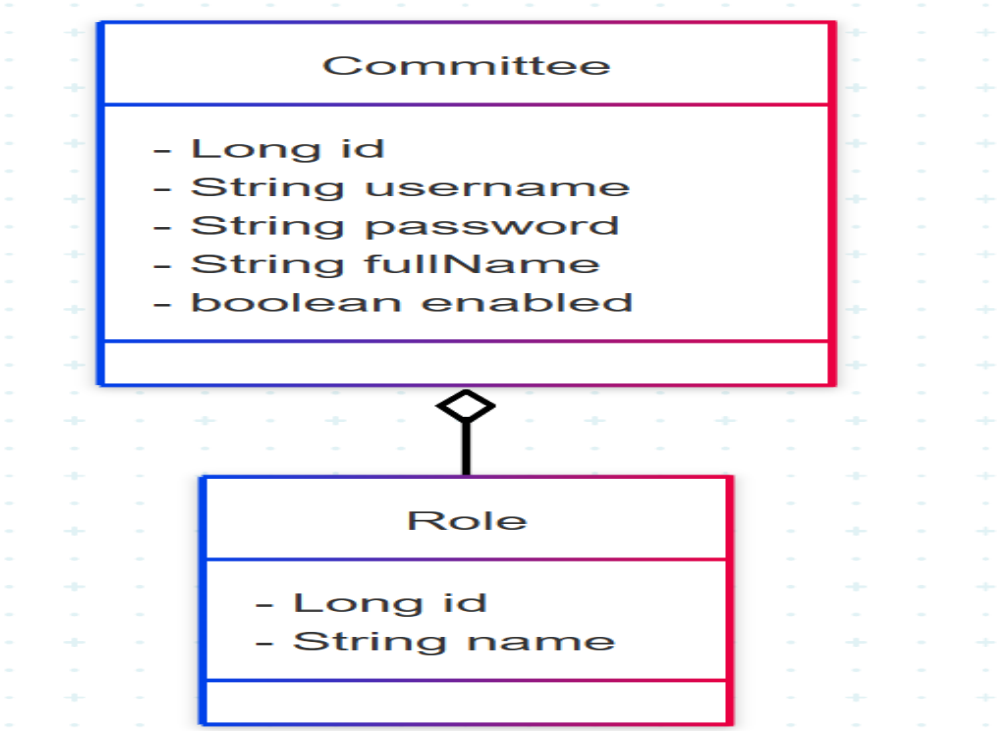
Class Name: Student	
Responsibilities: ▪ Represents a student's data ▪ Maintains information such as name, username, interests and skills ▪ Associated with his/her role in the system (e.g. STUDENT) ▪ Can have Applications, Logbooks and Evaluations	**Collaborations:** ▪ Role ▪ Application ▪ Logbook ▪ Evaluation ▪ TraineeshipPosition



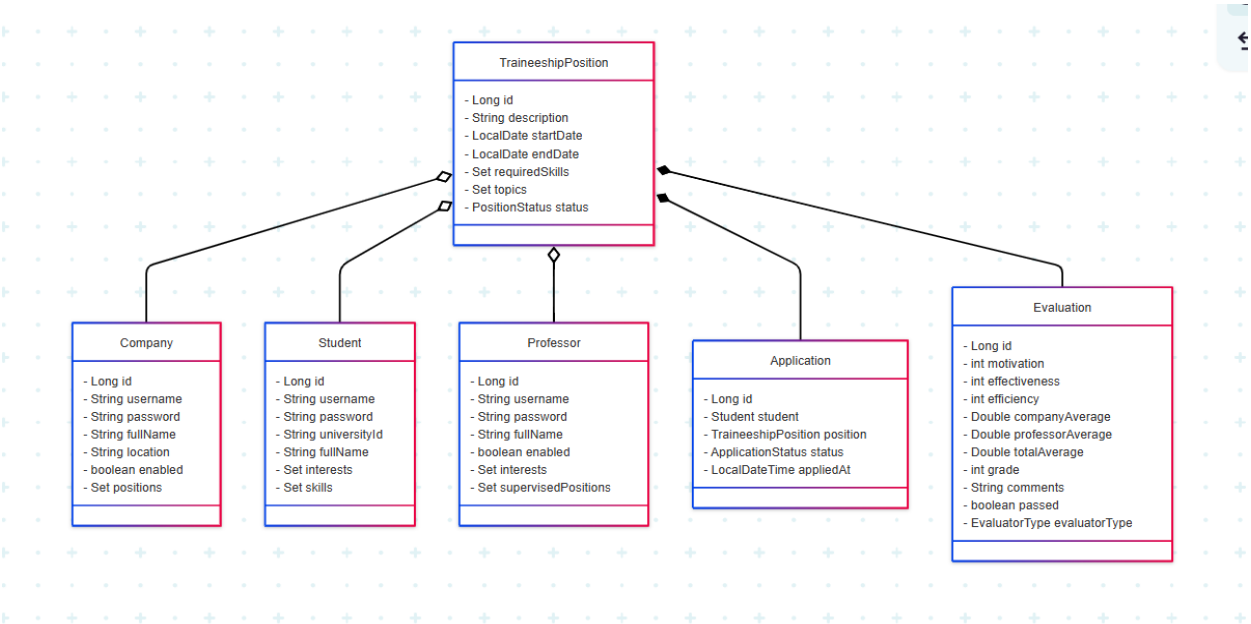
Class Name : Professor	
Responsibilities: ▪ Represents a professor who supervises internships ▪ Has username, password and interests ▪ Associated with his/her role in the system (e.g. PROFESSOR) ▪ Supervises one or more traineeships (TraineeshipPosition)	**Collaborations:** ▪ Role ▪ TraineishipPosition



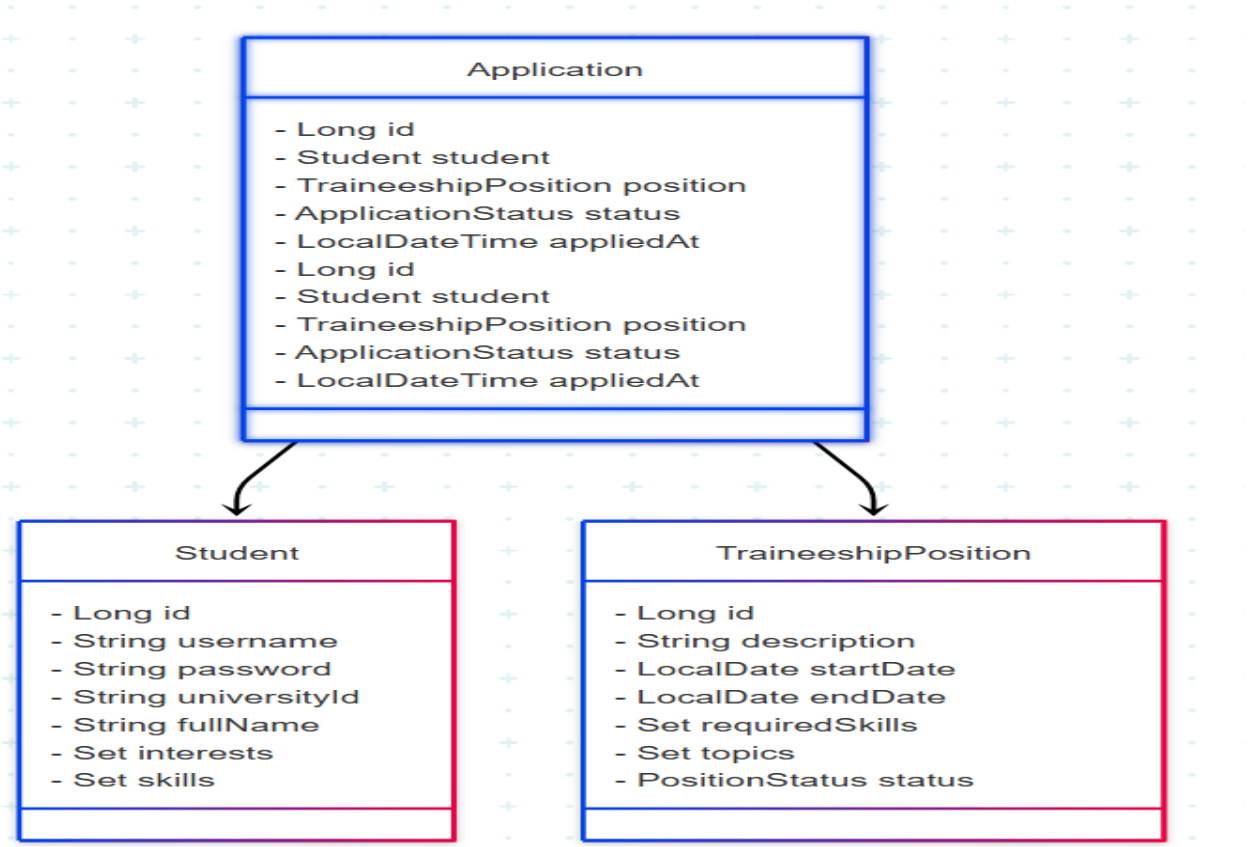
Class Name: Company	
Responsibilities: ▪ Represents a company that offers internships ▪ Contains details such as username, fullName, location, activation status ▪ Can post multiple Traineeship Positions	**Collaborations:** ▪ Role ▪ TraineeshipPosition



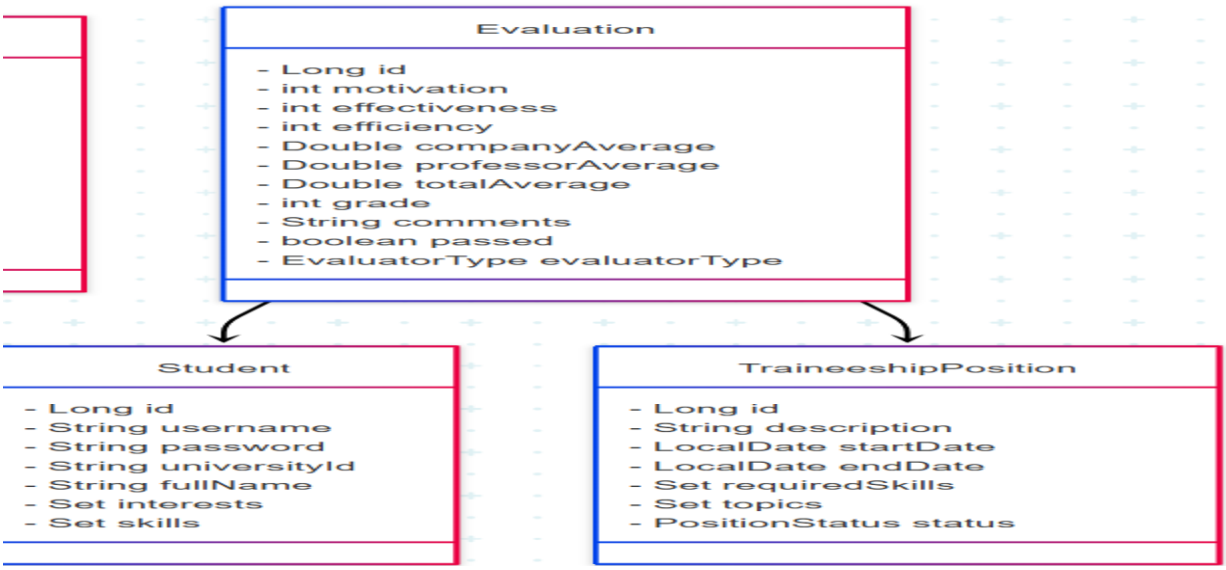
Class Name: Committee	
Responsibilities: ▪ Represented by a member of the Committee ▪ Contains access details and full name ▪ Activated/deactivated depending on access	**Collaborations:** ▪ Role



Class Name:TraineeshipPosition	
Responsibilities: ▪ Represents a place of traineeship positions ▪ Includes information such as description, dates, required skills and topics ▪ Has a position status (e.g. ***AVAILABLE***, ***ASSIGNED***, ***COMPLETED***)	**Collaborations:** ▪ Class Company (provide the location) ▪ Class Professor (possibly supervisor) ▪ Class Student (candidate/selected) ▪ Class Application (applications for the post) ▪ Class Evaluation (evaluations on the position) ▪ Enum PositionStatus(possible status states)



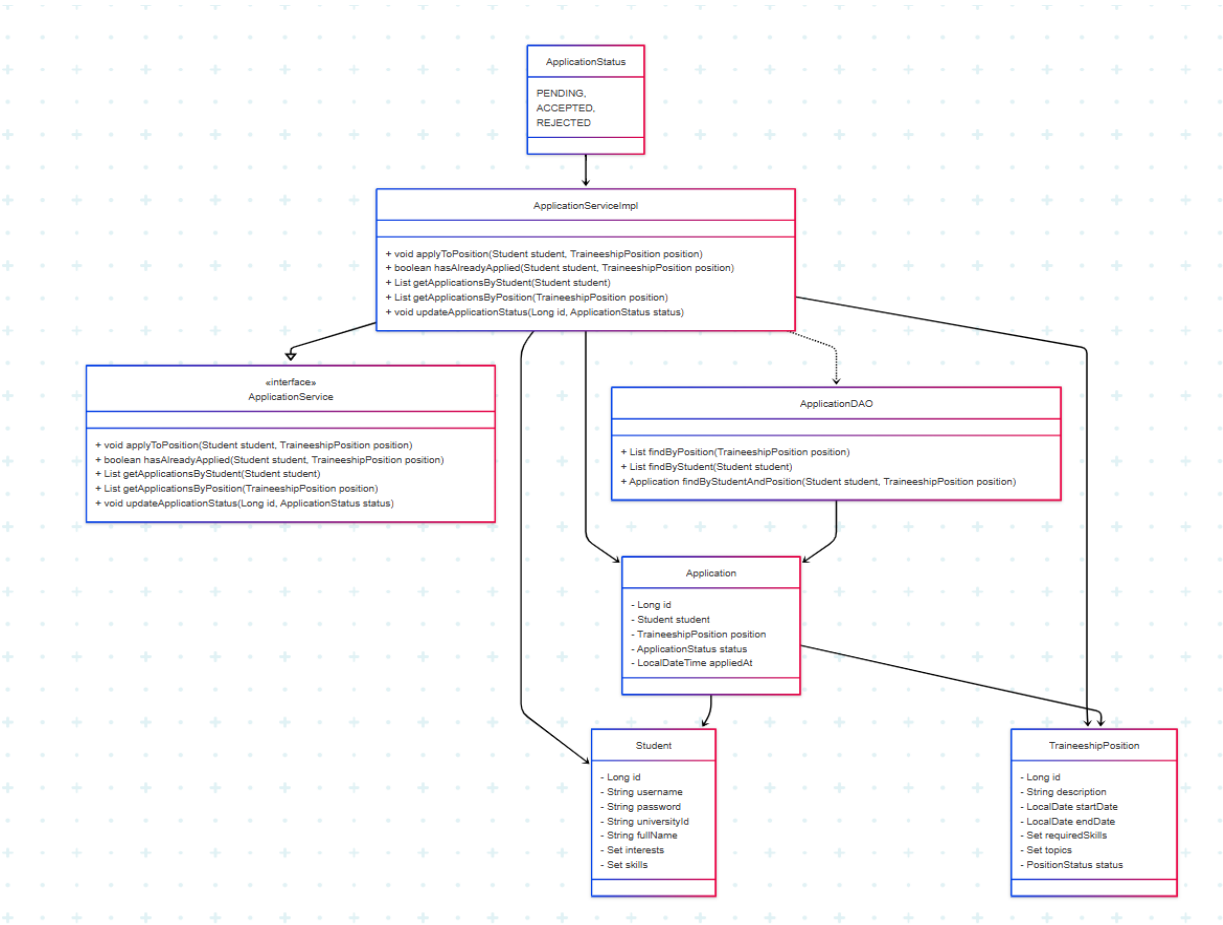
Class Name: Application	
Responsibilities: ▪ Represents a student's application for an internship ▪ Contains the application date and current status (e.g. PENDING, ACCEPTED)	**Collaborations:** ▪ Class TraineeshipPosition ▪ Class Student



Class Name: Evaluation	
Responsibilities: ▪ Evaluates the student in relation to the internship position ▪ Contains individual criteria (motivation, effectiveness, efficiency) ▪ Includes averages and overall grade ▪ Determines whether the internship was passed	**Collaborations:** ▪ Student (the student being assessed) ▪ TraineeshipPosition (the position in which the student is being assessed)

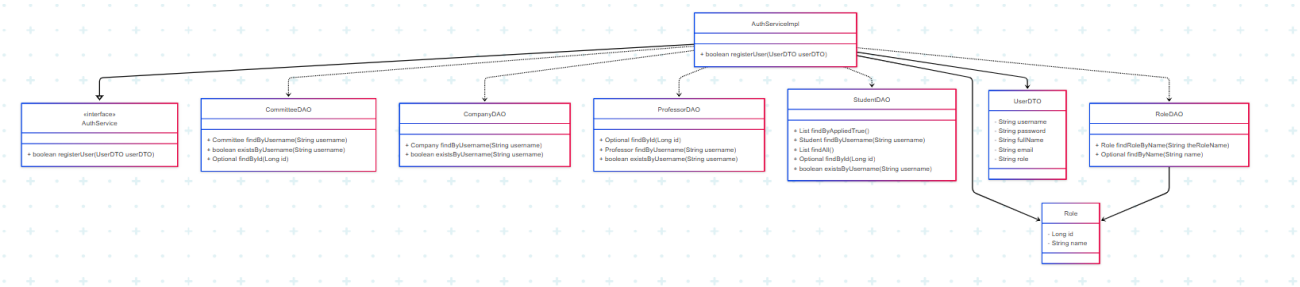
Service UML'S

"To save space, we haven't written every class's methods in detail, although they are mentioned in the UML's above."

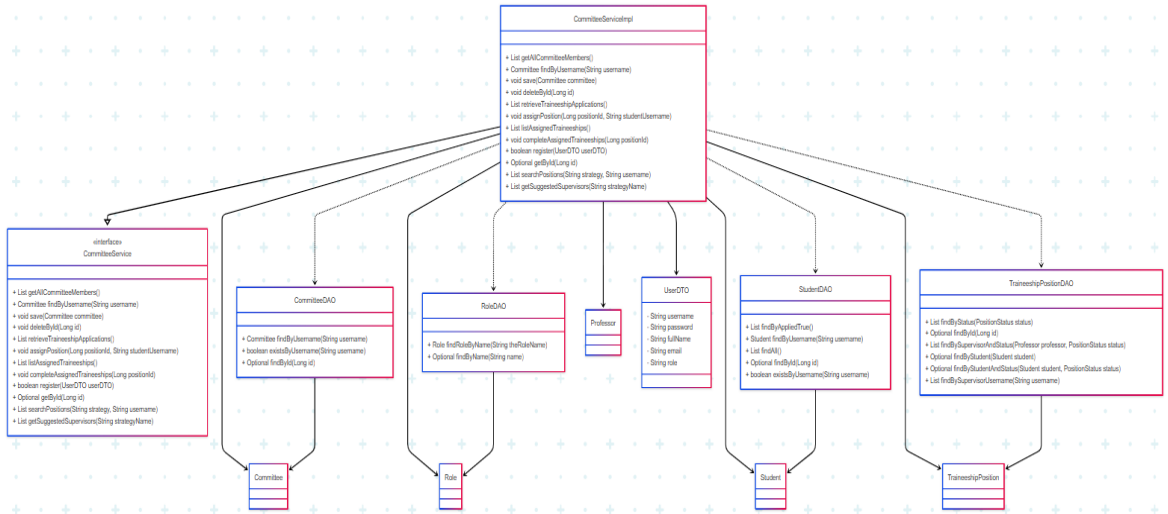


Class Name: Applcation ServiceImpl	
Responsibilities: ▪ Implements the logic of applying for a traineeship ▪ Checks if there is already an application from a student ▪ Updates the application status	**Collaborations:** ▪ ApplicationDAO (for storing and retrieving applications) ▪ TraineeshipPositionDAO (for access to positions) ▪ Application, Student,

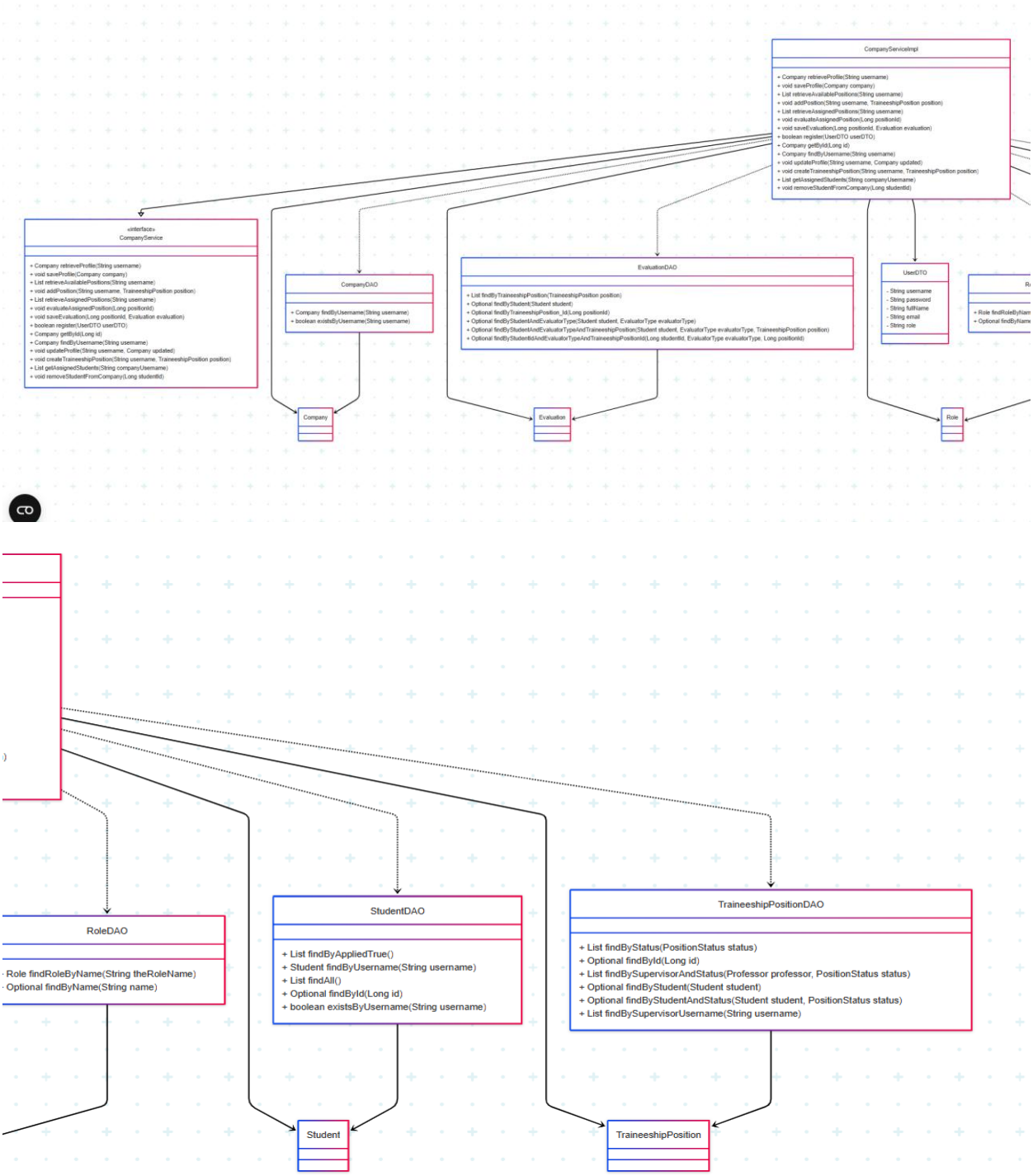
▪ Returns lists of applications by student or position	TraineeshipPosition (models) ▪ ApplicationService (implementing interface)



Class Name: AuthServiceImpl	
Responsibilities: ▪ Implements the user registration method ▪ Depending on the role, creates objects of type Student, Professor, Committee, Company ▪ Stores the objects in the corresponding databases via DAO	**Collaborations:** ▪ UserDTO (registration data) ▪ RoleDAO (find role) ▪ StudentDAO, ProfessorDAO, CompanyDAO, CommitteeDAO (storage of corresponding entities) ▪ Role, Student, Professor, Company, Committee (entities created)

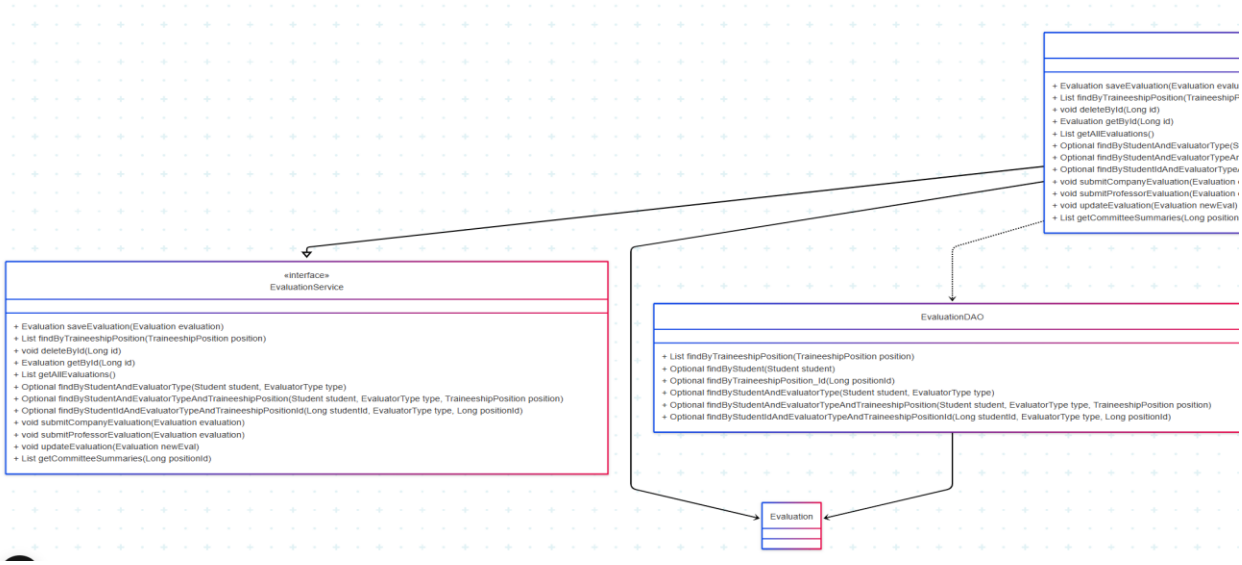


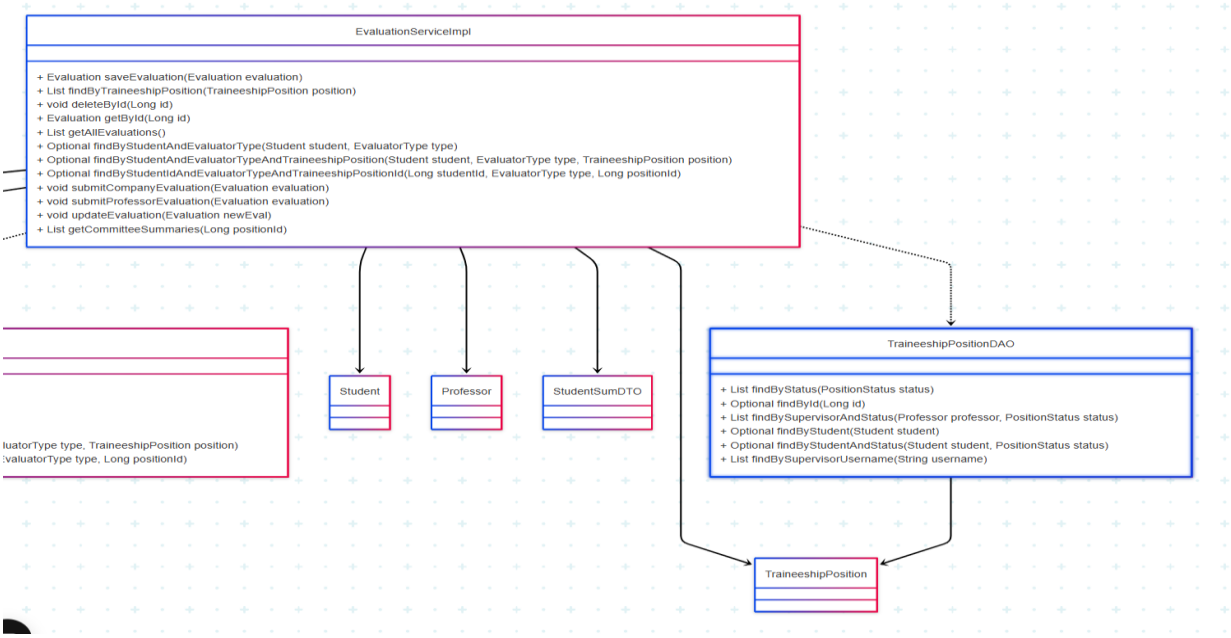
Class Name: CommitteeServiceImpl	
Responsibilities: ▪ Manages committee members ▪ Registers new members. ▪ Assigns internships to students ▪ Returns positions per appropriate strategy ▪ Recommends supervisors	**Collaborations:** ▪ CommitteeDAO ▪ RoleDAO ▪ StudentDAO ▪ TraineeshipPositionDAO ▪ UserDTO ▪ Committee, Role, Student, Professor, TraineeshipPosition(models)



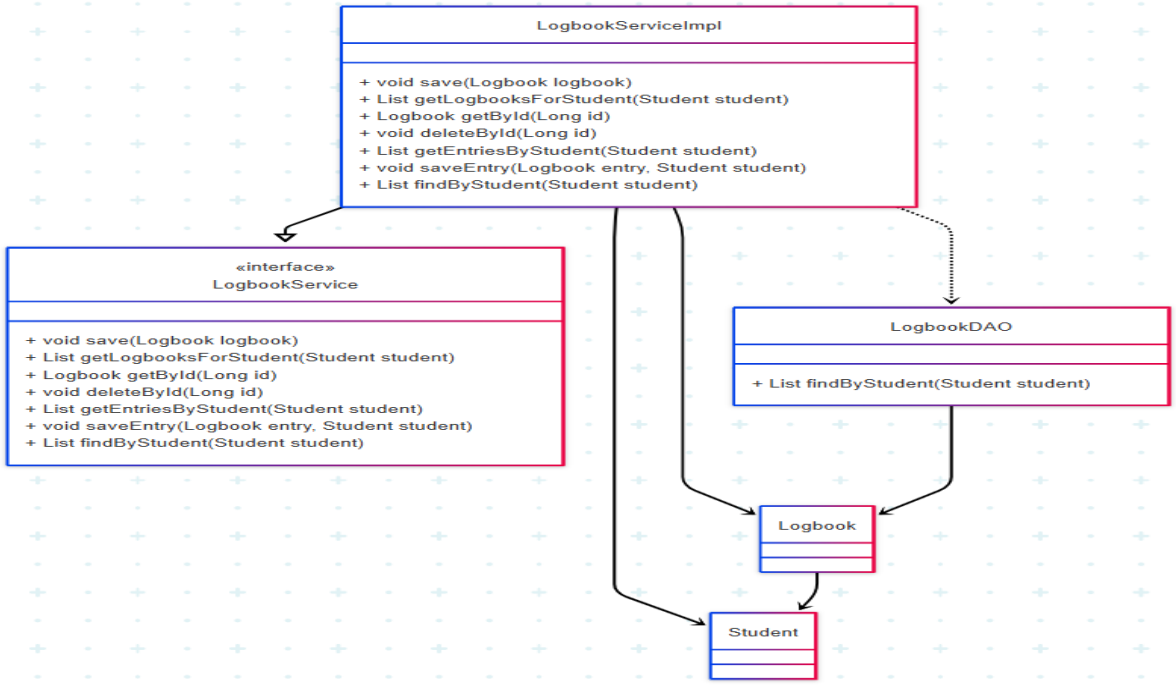
Class Name: CompanyServiceImpl	
Responsibilities: ▪ Manages company profiles and internship positions	**Collaborations:** ▪ CompanyDAO

▪ Registers new company in the system ▪ Creates and retrieves internships ▪ Creates new placements and creates new internships	▪ EvaluationDAO ▪ RoleDAO ▪ StudentDAO ▪ TraineeshipPositionDAO ▪ Company, Evaluation, Student, TraineeshipPosition, Role ▪ ....

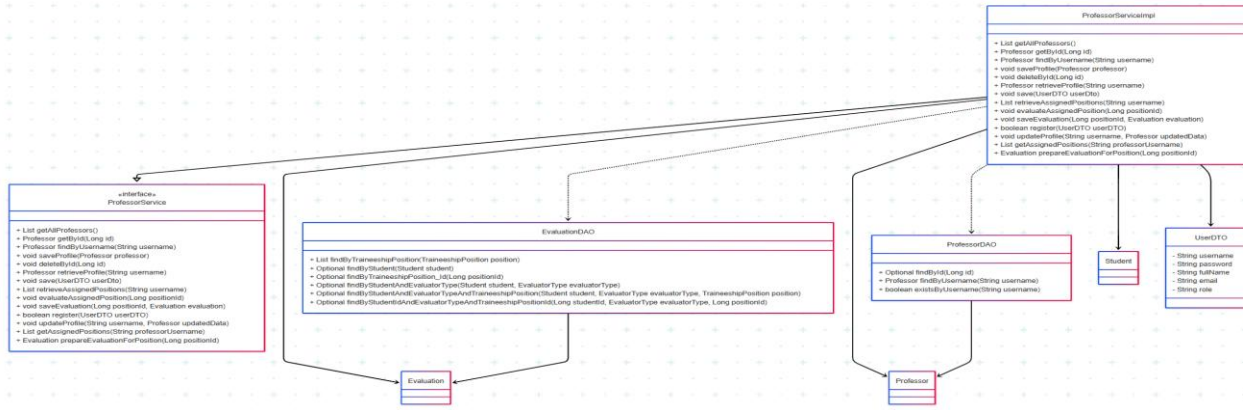


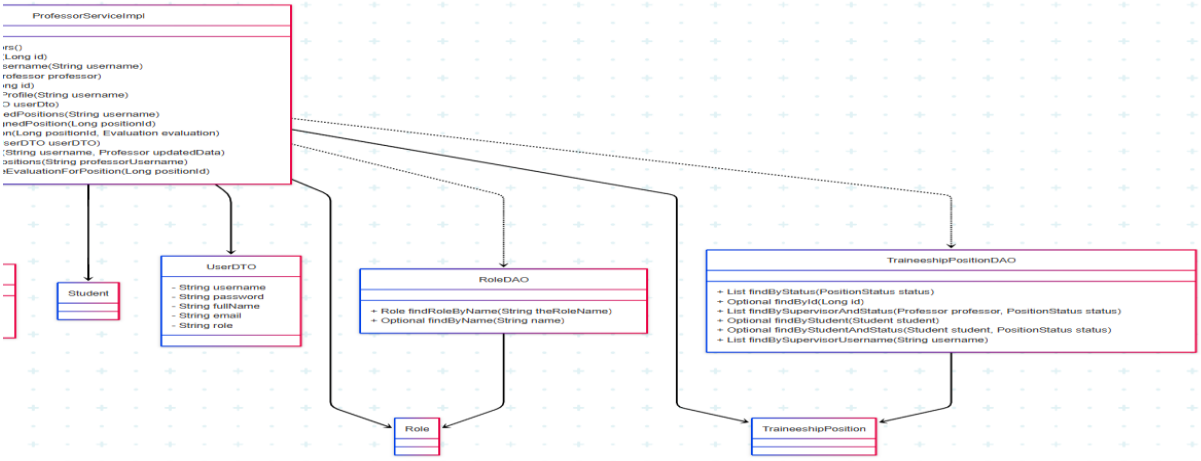


Class Name: EvaluationServiceImpl	
Responsibilities: ▪ Stores and updates assessments ▪ Submits company and teacher evaluations ▪ Returns evaluations for students and positions ▪ Provides summary of evaluations to committee	**Collaborations:** ▪ EvaluationDAO ▪ TraineeshipPositionDAO ▪ Evaluation, Student, Professor, TraineeshipPosition, StudentSumDTO

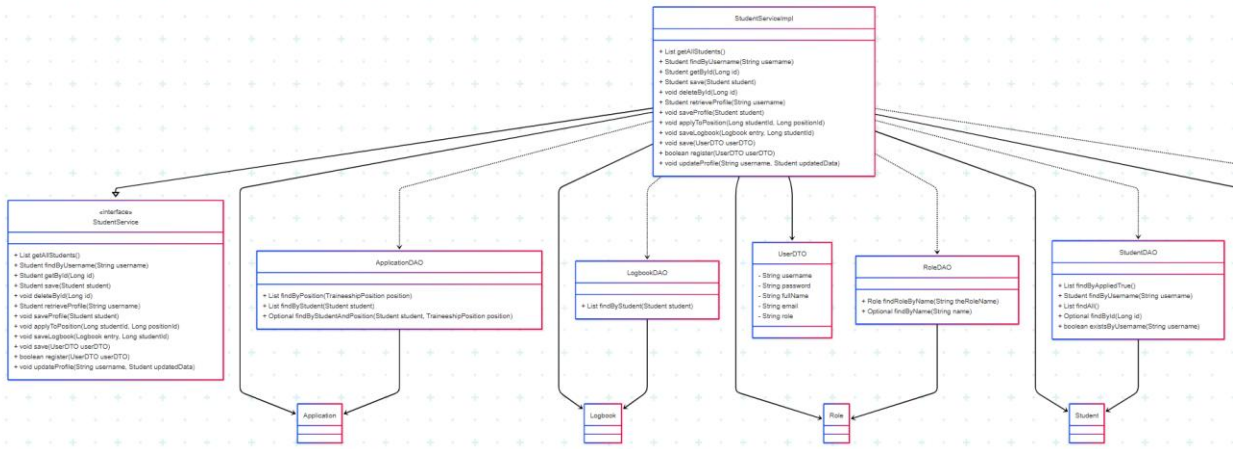


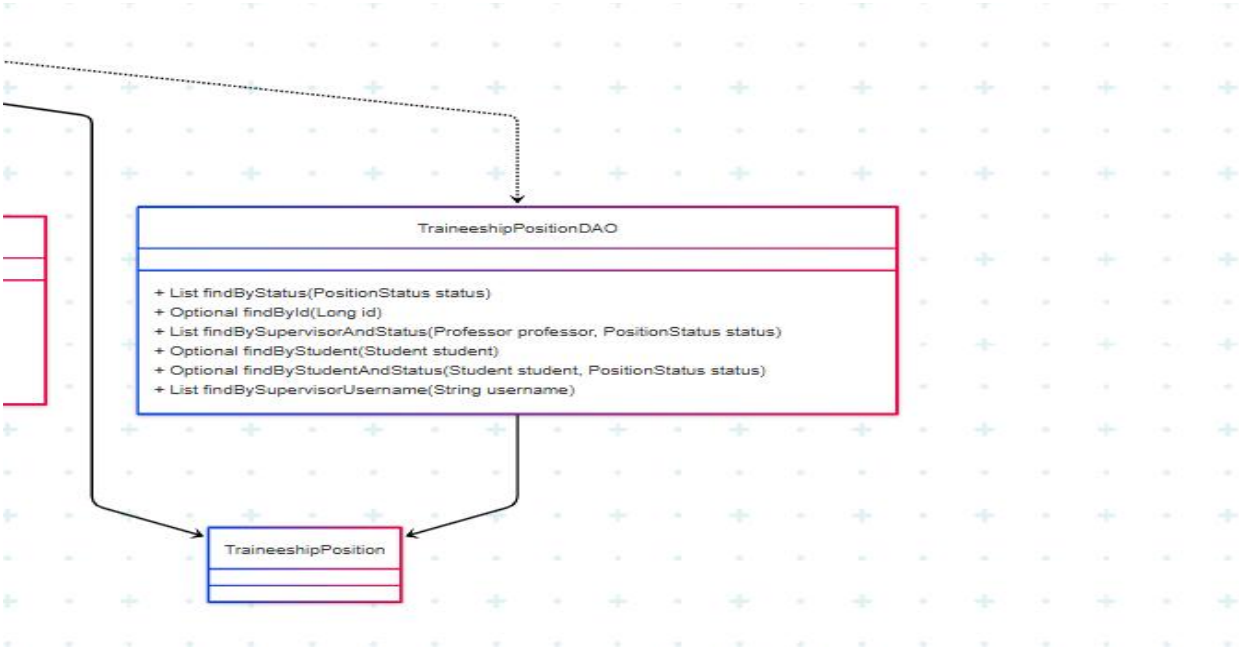
Class Name: LogbookServiceImpl	
Responsibilities: ▪ Manages practice calendar entries ▪ Stores, deletes and retrieves entries for students ▪ Finds all entries per student	**Collaborations:** ▪ LogbookDAO ▪ - Logbook ▪ - Student ▪ ....





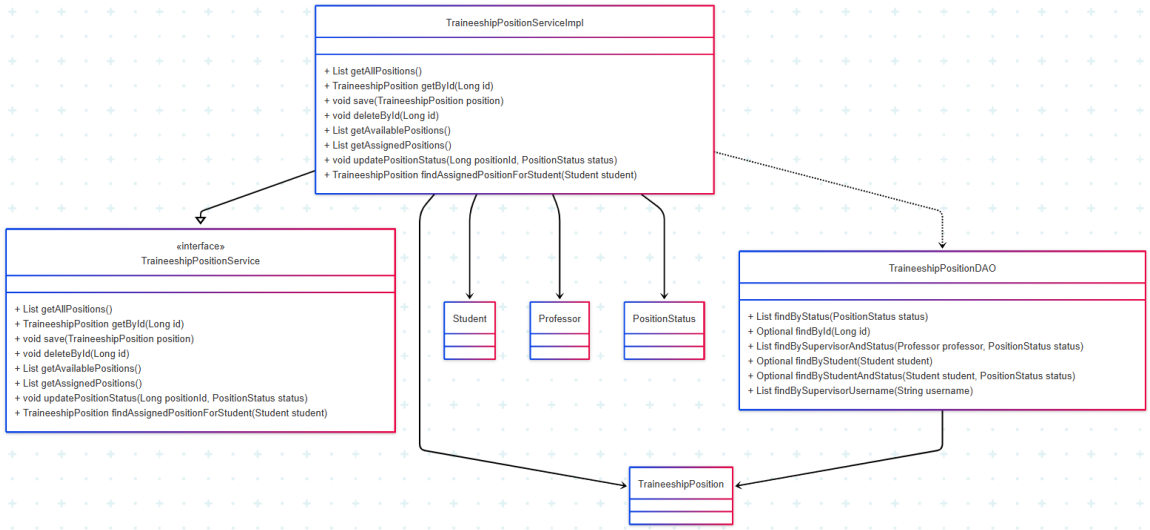
Class Name: ProfessorServiceImpl	
Responsibilities: ▪ Manages the teacher profile ▪ Assigns and evaluates internships ▪ Assesses and assigns placements and assigns placements ▪ Provides access to supervised positions	**Collaborations:** ▪ EvaluationDAO ▪ ProfessorDAO ▪ RoleDAO ▪ TraineeshipPositionDAO ▪ Professor, Evaluation, Student, Role, TraineeshipPosition





Class Name: StudentServiceImpl	
Responsibilities: ▪ Manages the student profile ▪ Enables registration, editing and retrieval of data ▪ Allows applications for internships and logbook creation	**Collaborations:** ▪ ApplicationDAO ▪ LogbookDAO ▪ RoleDAO ▪ StudentDAO ▪ TraineeshipPositionDAO

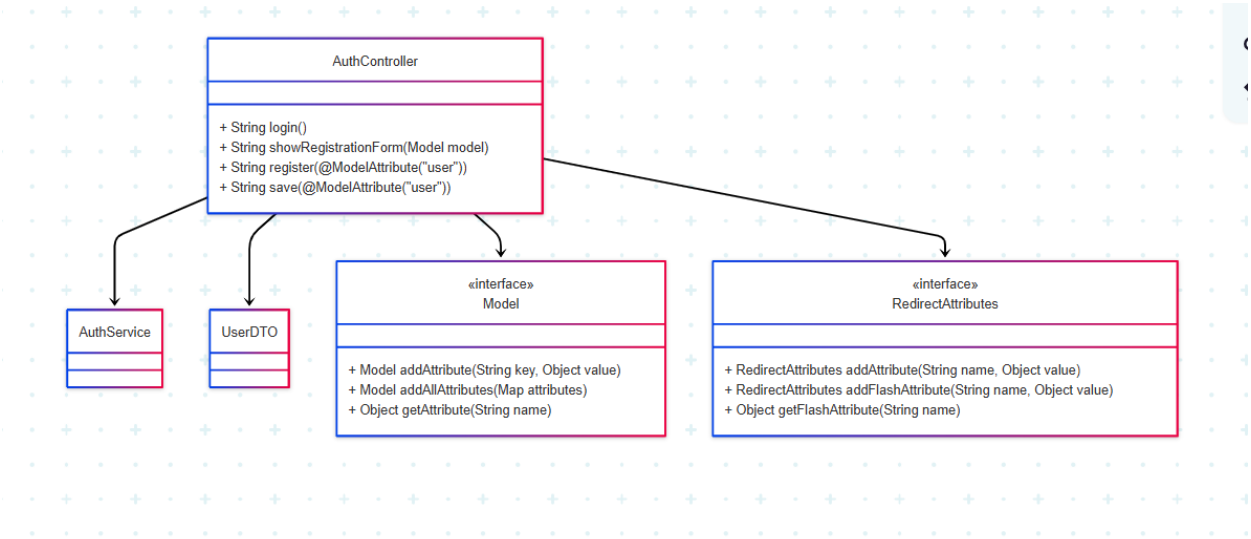
	▪ Student, Application, Logbook, TraineeshipPosition, Role



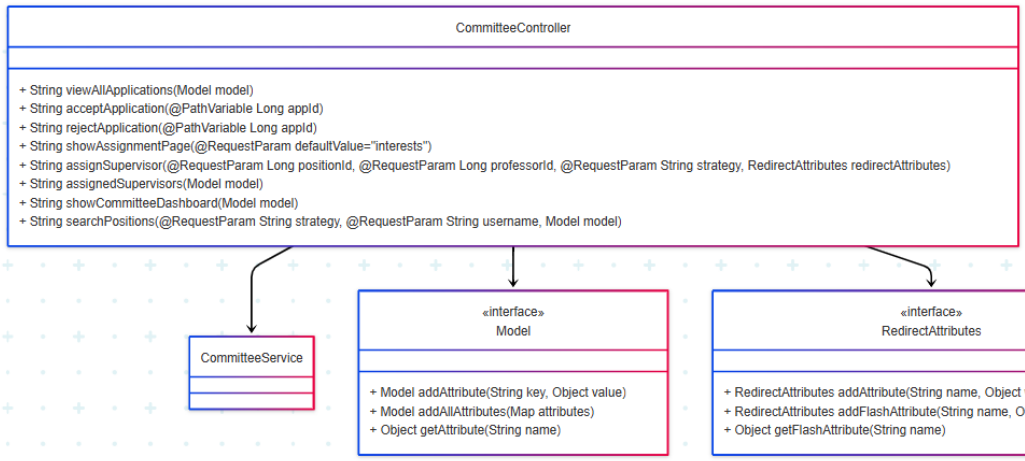
Class Name: TraineeshipPositionServiceImpl	
Responsibilities: ▪ Manages all internship positions ▪ Manages internship status assigned or available ▪ Finds positions based on status or student ▪ Stores, updates and deletes positions	**Collaborations:** ▪ TraineeshipPositionDAO ▪ TraineeshipPosition ▪ Student ▪ Professor ▪ PositionStatus

Controller UML'S

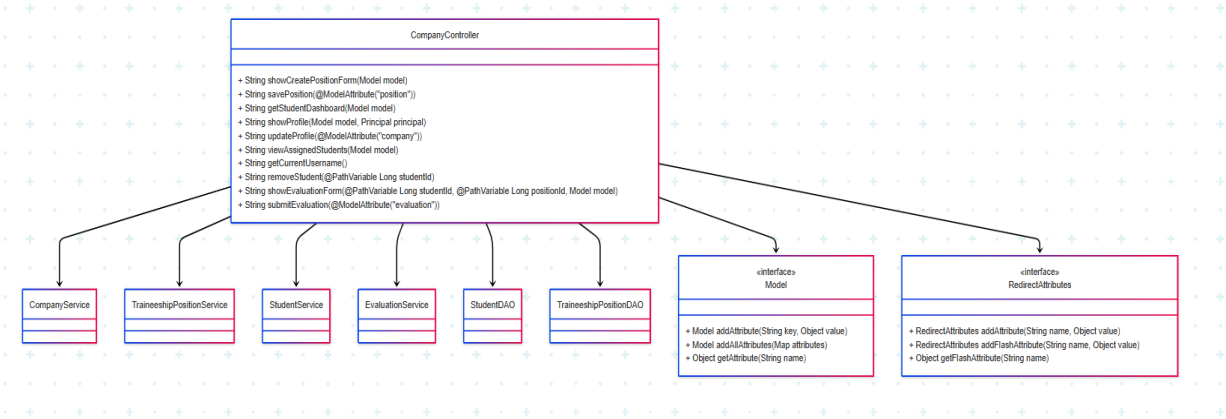
"To save space, we haven't written every class's methods in detail, although they are mentioned in the UML's above."



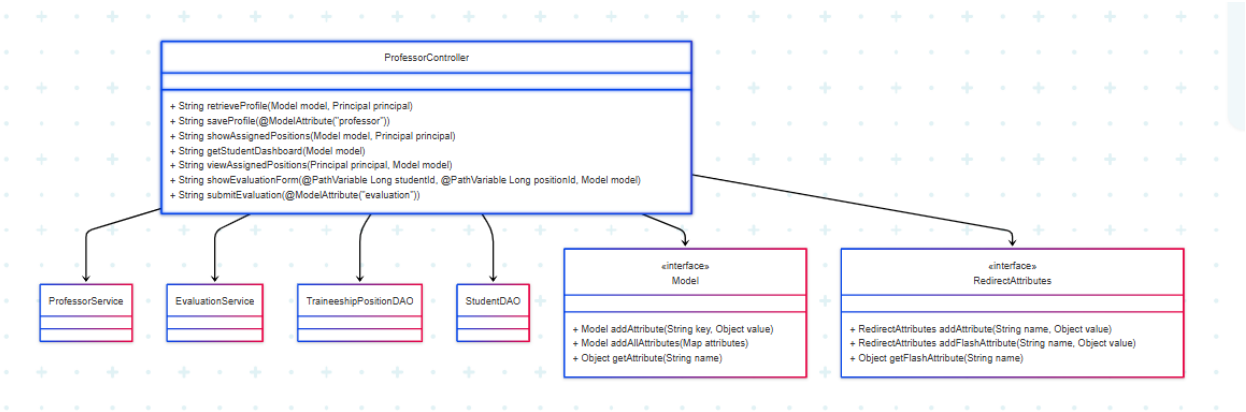
Class Name: AuthController	
Responsibilities: ▪ Operates the user login and registration form ▪ Forwards data to AuthService for registration processing	**Collaborations:** ▪ AuthService (logical registration processing) ▪ UserDTO (object with registration data) ▪ Model (to push data to view) ▪ RedirectAttributes (to pass messages during redirect)



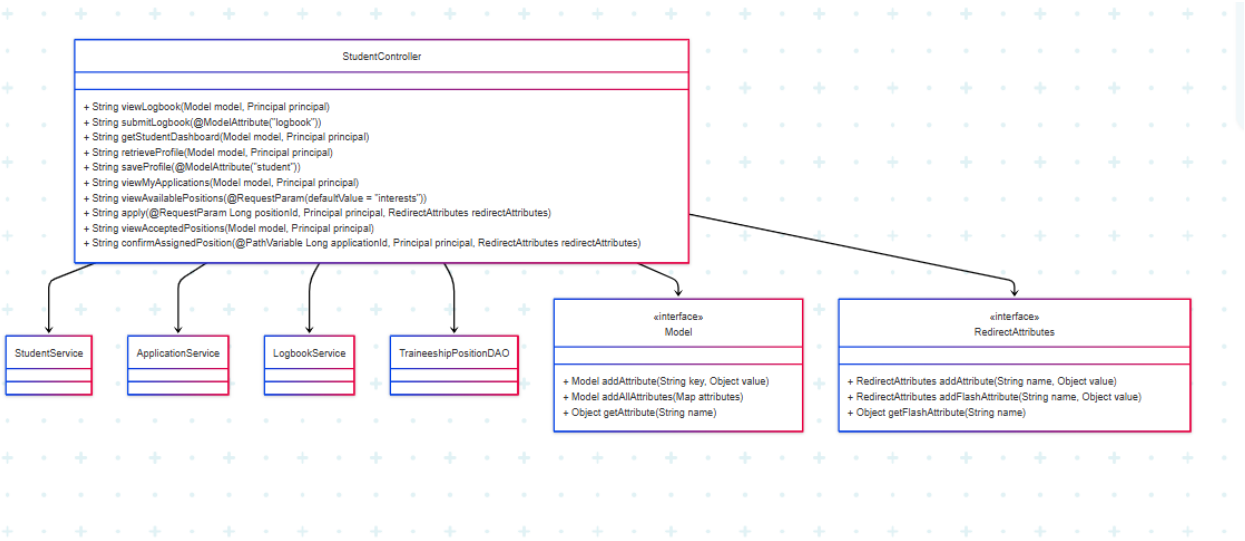
Class Name: CommitteeController	
Responsibilities: ▪ Present applications to the committee ▪ Allows acceptance or rejection of applications ▪ Assigns supervisors to positions ▪ Supports search and viewing of statistics ▪ Displays the committee dashboard	**Collaborations:** ▪ CommitteeService (business logic) ▪ Model (data transfer to views) ▪ RedirectAttributes (message forwarding during redirect)



Class Name: CompanyController	
Responsibilities: ▪ Creates and stores practice positions ▪ Displays and updates company profiles ▪ Displays assigned students ▪ Submits student evaluations ▪ Shows control table ▪ Displays the company's dashboard	**Collaborations:** ▪ ApplicationService (for managing applications and assessments) ▪ Model (for transmitting data to views) ▪ RedirectAttributes (for message redirects)



Class Name: ProfessorController	
Responsibilities: ▪ Displays and saves teacher profiles ▪ Provides assigned internships ▪ Provides student evaluation form ▪ Submits evaluations ▪ Displays the professor's dashboard	**Collaborations:** ▪ ProfessorService (main business logic) ▪ EvaluationService (evaluation submission) ▪ TraineeshipPositionDAO (retrieve assigned positions) ▪ StudentDAO (student information) ▪ Model (to views) ▪ RedirectAttributes (flash messages)



Class Name: StudentController	
Responsibilities: ▪ Displays the student dashboard and logbook ▪ Manages internship applications ▪ Edits the student profile ▪ Provides access to available positions	**Collaborations:** ▪ StudentService ▪ ApplicationService ▪ LogbookService ▪ TraineeshipPositionDAO ▪ model ▪ RedirectAttributes